Expernetic Library Application – Project Report

Software-Engineer Internship Assignment

Binuka Senadheera

Dec / 2025

---

1. Project Overview

This project implements a full-stack CRUD application for managing a collection of books.

The system enables users to create, read, update, and delete book records through a RESTful API and a React-based user interface.

The backend is developed using **ASP.NET Web API** with **Entity Framework Core** and **SQLite**, while the frontend is built with **React and TypeScript**.

The design emphasizes clarity, maintainability, and correct handling of CRUD operations.

---

2. Development Process

2.1 Backend Implementation

The backend follows a clean folder structure to clearly separate concerns:

- **Models** – define the domain entities
- **Data** – contains the DbContext and database configuration
- **Controllers** – expose API endpoints
- **Migrations** – store EF Core-generated migration files

The application adopts a **code-first approach**.

A Book entity defines the necessary fields, such as:

- Id
- Title
- Author
- Description

This entity maps to the SQLite database through Entity Framework Core. A LibraryContext class manages database access. Migration commands are used to generate the schema and update the SQLite database file.

API Design

The API exposes the following REST endpoints:

- **GET /api/books** – retrieves all books
- **GET /api/books/{id}** – retrieves a single book by ID
- **POST /api/books** – creates a new book entry
- **PUT /api/books/{id}** – updates an existing book
- **DELETE /api/books/{id}** – removes a book

Each endpoint uses the DbContext to perform database operations and returns appropriate HTTP responses.

Error Handling and Validation

The backend includes validation to ensure required fields are present and uses standard HTTP status codes to indicate error cases.

Operations such as retrieving or updating a non-existent record return meaningful responses to ensure predictable API behavior.

2.2 Frontend Implementation

The frontend is implemented using **React with TypeScript**.

It interacts with the backend through HTTP requests and presents a clean interface for managing books.

Key Features

- Displays a list of books retrieved from the API
- Provides forms for adding and editing book entries
- Updates the interface immediately after operations
- Supports deletion of entries

A small set of components organizes the UI logic:

- A list view component displays existing records
- A form component handles create and update actions
- An API utility module centralizes HTTP calls

State Management

React's useState and useEffect hooks manage component state and data loading.

Given the project's scope, complex state management libraries are unnecessary.

3. Running the Application

Clone the Git Repository

1. Navigate to the api directory.
2. Run:

```
git clone https://github.com/BinukaDs/Expernetic-Assignment.git
```

Backend

3. Navigate to the api directory.
4. Run:

```
dotnet restore
dotnet ef database update
dotnet run
```

3. The API becomes accessible at http://localhost:<port>.

Frontend

1. Navigate to the frontend directory.
2. Run:

```
npm install
npm run dev
```

3. The application loads in the browser at the provided local development URL.

---

4. Challenges and Considerations

The project required integrating multiple technologies, including ASP.NET, EF Core, SQLite, and React.

though I have previous experience working with React and Node.js, I'm a beginner when it comes to .NET. This project required me to gain experience with C#, the .NET Core Web API architecture, and Entity Framework Core for database operations. I spent time reading the documentation and exploring YouTube tutorials to understand building APIs in the .NET ecosystem.

Addressing these challenge gained me experience of full-stack application architecture and data flow.

---

5. Additional Enhancements

The application includes several improvements beyond basic CRUD requirements:

- A clean, Git repo maintenance using Gitflow strategy
- Clear error feedback for invalid actions
- A clean, maintainable folder structure
- Multiple Book Deletion

---

6. Conclusion

This project demonstrates the complete workflow of building a full-stack CRUD application. It covers backend API design, database integration with EF Core and SQLite, and frontend interaction using React and TypeScript. The final system is stable, well-structured, and fulfills all core CRUD requirements while remaining easy to extend or modify.